

Circadia Risk & Compliance Engines - Technical Overview

Document date: 8 July 2026

Status: Reference for product and engineering development

Repository: fatigue-app-latest (app-next + frms-engine)

Audience: Architecture review, further engine development

Executive summary

Circadia implements three separate engine families on a shared minute-grid diary foundation. They must not be conflated:

Engine | Question | Legal weight

|-----|-----|-----|

Compliance (retrospective) | Did the attested record breach WA Reg 184E? | Violations/warnings on logged time

Prospective risk (ISO 31000) | If the declared run plan proceeds, what might happen? | Coaching only - not enforcement

FRMS fatigue assurance | How fatigued is the driver biomechanically / heuristically? | Assurance glance - not statutory, not NHVR FRMSc

Command (circadia-command) has no compliance or risk engines - incident triage only.

Part 1 - Shared foundation: minute timeline

All engines consume driver events (work / break / stop / non-work) converted to 1440-minute boolean grids per calendar day.

Module | Role

|-----|-----|

coverage/derive-minute-coverage.ts | Events -> work_time[], non_work[], breaks[]

five-hour-break-rule.ts | Rolling 5h work window + qualifying rest slots

rolling-events.ts | Rolling timeline without midnight boundaries

compliance-history.ts | Up to 12 prior weeks for 168h / 28d context

Coverage rules (product mapping to Reg 184E):

- Break < 10 min -> counts as work
 - Break >= 10 min -> can fill rest slots
 - Break > 30 min -> treated as non-work (full rest block)
-

Part 2 - Compliance engine (retrospective)

Entry: getComplianceEngine(jurisdiction) -> waOsh3132Engine -> runComplianceChecks() in compliance.ts

NHVR: nhvr-provisional-engine.ts runs the same WA math plus a disclaimer banner. Separate NHVR BFM rule pack is not implemented.

2.1 Five-hour break rule

Within each rolling 300 work minutes, qualifying rest is:

- One continuous break ≥ 20 min, OR
- Two separate breaks each ≥ 10 min

Algorithm (slot model):

```
slot1, slot2 <- empty
for each break segment in rolling 5h window:
  if duration  $\geq 20$  -> slot1 = slot2 = true
  else if duration  $\geq 10$  -> fill slot1, then slot2
complete when slot1 AND slot2
```

Constants: WORK_WINDOW_MIN = 300, MIN_QUAL_BREAK_MIN = 10, TOTAL_QUAL_BREAK_MIN = 20

2.2 Rolling 168h / 14-day work limit

Shared with prospective risk via rolling-168h-metrics.ts (must stay aligned with compliance.ts).

Segmentation: Split timeline at ≥ 48 h continuous non-work (resets rolling 14-day window).

Within each segment, for every rolling 14 x 24 x 60 minute window:

```
W_max = max over start of sum(work[i] for i in [start, start + 20159])
```

Condition | Result

```
|-----|-----|
```

$W_{\max} > 168 \times 60$ minutes | Violation: "14-day work exceeds 168h"

$W_{\max} > 140 \times 60$ minutes | Warning: approaching 168h

Headroom | $168 - W_{\max}/60$ hours

Exported constants: MAX_WORK_HOURS_14D = 168, WARN_WORK_HOURS_14D = 140

2.3 Solo driver rules (Reg 184E)

72h window ending at "now":

- ≥ 27 h total non-work
- ≥ 3 blocks of ≥ 7 h continuous non-work
- Between consecutive ≥ 7 h blocks: elapsed non-rest time ≤ 17 h (violation if exceeded)

14-day / 28-day non-work:

- 14d option: $\geq 2 \times 24$ h continuous non-work in 14 days
- 28d alternative: four 24h blocks plus no rolling 14d work window > 144 h inside 28d

2.4 Two-up driver rules

Check | Threshold

|-----|-----|

Rolling 24h | ≥ 7 h non-work

7-day option | ≥ 48 h non-work, includes ≥ 24 h continuous, no block < 7 h

48h option (if 7-day not met) | $\geq 1 \times 7$ h non-work not in moving vehicle (GPS heuristic)

2.5 Shift change (Reg 184E(4))

Pattern A/B alternation with ≥ 24 h gap between shift changes (shift-change.ts).

2.6 Compliance outputs

```
ComplianceCheckResult {
  type: "violation" | "warning"
  message: string
  ruleId?: string
  day?: number
}
```

Product policy: CONCURRENT_COMPLIANCE.md - warn/report, never block lodging.

Part 3 - Prospective risk engine (ISO 31000 / IEC 31010)

Purpose: Score future run plans only - never re-label logged minutes as "risk".

Module | Role

|-----|-----|

compliance-state.ts | complianceStateAt(asOf) -> rolling 168h headroom at now

risk-scenarios.ts | Inject planned hours on future days; ± 2 h sensitivity

risk-evaluate.ts | Likelihood x consequence matrix

risk-register.ts | Orchestrator -> register entries

risk-criteria.ts | Bands and thresholds

ADR: app-next/docs/adr/0003-prospective-risk-engine.md

3.1 Non-overlap rule

Domain | Time | Question

|-----|-----|-----|

Compliance | Past + present (logged) | Did we breach?

Risk | Future (planned, not logged) | If plan holds, what happens?

When a day is logged, it exits risk and enters compliance exclusively.

3.2 Scenario construction

For each future day with a run plan:

1. Baseline = complianceStateAt(now) from attested history
2. Inject planned on-duty hours from minute 0 of future day(s)
3. Scenarios: "planned" and "high" (+2h via RISK_HOURS_SENSITIVITY_DELTA = 2)
4. If only km given: hours = max(4, km/50)

3.3 Risk matrix formulae

Likelihood L from 168h headroom after scenario:

Condition	L
----- ---	
Would exceed 168h	5
Headroom < 0	5
Headroom < 12h	4
Headroom < 24h	3
Headroom < 48h	2
Else	1

Consequence C:

Condition	C
----- ---	
Would exceed 168h	4
In warning band (140-168h)	2
Else	1

Scores:

```

RiskScore = L x C
ResidualLikelihood = max(1, L - 1) // assumes standard barriers
ResidualScore = ResidualLikelihood x C

```

Risk bands (scoreToRiskLevel):

Score	Level
----- -----	
>= 21	critical
>= 15	elevated
>= 7	monitor
< 7	low

Standard barriers (assumed for residual risk):

- Recorded breaks and End shift
- 7h non-work before Start shift

- Revise run plan or add rest day

Part 4 - FRMS fatigue assurance

Selected by FRMS_ENGINE environment variable (legacy default, python for TPMA).

4A. Legacy TypeScript sawtooth (fatigue-risk-carry.ts)

Time-on-task carry c in $[0,1]$, updated per 15-min block:

```
if nonWork OR recovery >= 30 min:
  c <- c x 0.12      // ~88% relief
else if recovery >= 15 min:
  c <- c x 0.40      // ~60% relief
else if workMinutes > 0:
  c <- min(1, c + workMinutes / 300) // saturates at ~5h continuous work
```

Key constants:

- FATIGUE_TIME_ON_TASK_SATURATION_MIN = 300
- FATIGUE_PARTIAL_BREAK_RECOVERY_MIN = 15
- FATIGUE_FULL_REST_RECOVERY_MIN = 30
- FATIGUE_POST_PARTIAL_BREAK_MULTIPLIER = 0.4
- FATIGUE_POST_FULL_REST_MULTIPLIER = 0.12

Composite fatigue index (manager-risk-timeline.ts), without camera:

$$I = 0.40xc + 0.18xC_{\text{circ}} + 0.10xW_{\text{load}} + 0.14xA_{14d} + 0.06xD_{\text{plan}} + 0.08xS_{\text{self}}$$

Term | Meaning

|-----|-----|

c | Time-on-task carry

C_{circ} | Circadian (dual sine, local hour)

W_{load} | Work minutes in block / 15

A_{14d} | $\min(1, \text{rollingWorkHours}_{14d} / 168)$

D_{plan} | Plan deviation / 15

S_{self} | Self-report alertness 0-1

With camera: weights shift (28% camera term from drowsiness, distraction, eyes-off-road, yawns/nods).

Map to 0-100%:

$z = (I - \mu) / \sigma$ where $\mu = 0.38$, $\sigma = 0.20$ (default)
 $P = 100 / (1 + \exp(-\beta(z - \zeta)))$ where $\beta = 1.35$, $\zeta = 0$

UI thresholds: amber $\geq 45\%$, red $\geq 70\%$

4B. Python TPMA (frms-engine/app/math_engine.py)

Three-process model (Åkerstedt/Folkard) per 15-min block.

Process S (homeostatic pressure):

```
Deltas = chi_w x (1 + mu x h_continuous_work) // wake
Deltas = -chi_s x r_env // sleep/rest
```

Per 15-min block constants: $\chi_w = 0.035/4$, $\chi_s = 0.238/4$, $\mu = 0.015$ per continuous work hour.

Process W (sleep inertia) on wake:

```
w_t = W_0 x exp(-chi_i x t_wake) W_0 = 0.35, chi_i = 1.75/4
```

Process C (circadian) - two-harmonic alertness:

```
C_alert = clamp((A1xcos(2pi(t-phi1)/24) + A2xcos(2pi(t-phi2)/12) + A1+A2) / (2(A1+A2)), 0, 1)
A1 = 0.14, phi1 = 16.75, A2 = 0.04, phi2 = 14.50
```

Combined impairment %:

```
capacity = C_alert - s_t - w_t
impairment = 1 - (clamp(capacity, -0.5, 1.0) + 0.5) / 1.5
pct = round(impairment x 100)
```

Bands: ≤ 35 low, ≤ 54 monitor, ≤ 74 elevated, else critical

Self-report bump: levels 1-5 map to factors {0.05, 0.25, 0.5, 0.75, 1.0} x 28 points

Progressive compression thresholds (continuous work hours):

Hours | Band | Rationale

|-----|-----|-----|

5.5 | monitor | TPMA workload accumulation

7.0 | elevated | Dawson-Reid ~0.05% BAC equivalence

10.0 | critical | ~0.10% BAC parity

Environmental modifiers on rest recovery: daytime penalty 25%; extreme heat/cold reduce recovery rate.

Part 5 - Manager fusion layer

manager-risk-scoring.ts merges:

1. Compliance violations -> highest tier ("attention")
2. Prospective risk register -> elevated/monitor
3. Near-term flags (unsigned sheets, GPS gaps, etc.)

When FRMS_ENGINE=python, manager API may replace the TypeScript risk register with Python prospective register from cached FrmsProfileRun.

Live fusion (risk-block-ingest.ts): diary blocks + optional camera packets -> live % on 15-min grid.

Shift lane projection (manager-shift-lane-plans.ts): projects break-due from 5h rule + demo fatigue walk.

Part 6 - Data flow architecture

Driver events -> derive-minute-coverage -> 1440-min grids

```

|
+--> runComplianceChecks -> violations / warnings
|
+--> complianceStateAt -> buildRiskRegister (LxC matrix)
|
+--> buildFrmsTimelinePayload -> math_engine.py (TPMA)
|
+--> fatigue-risk-carry (legacy sawtooth) -> manager-risk-timeline

```

Manager layer: manager-risk-scoring merges compliance + risk + FRMS timeline

Cross-app env (Command is separate): Manager and Command share Neon; Command does not run compliance math.

Part 7 - Key file index

Topic | Path

|-----|-----|

Compliance core		app-next/src/lib/compliance.ts
Engine router		app-next/src/lib/jurisdiction/compliance-engine.ts
168h metrics		app-next/src/lib/rolling-168h-metrics.ts
5h break		app-next/src/lib/five-hour-break-rule.ts
Risk matrix		app-next/src/lib/risk-evaluate.ts, risk-criteria.ts
Risk register		app-next/src/lib/risk-register.ts
Sawtooth		app-next/src/lib/fatigue-risk-carry.ts
Manager timeline		app-next/src/lib/manager-risk-timeline.ts
Python TPMA		frms-engine/app/math_engine.py
FRMS orchestrator		app-next/src/lib/frms/orchestrator.ts
WA regulatory map		app-next/docs/regulatory/wa-commercial-vehicle-hours.md
ADR risk vs compliance		app-next/docs/adr/0003-prospective-risk-engine.md
Sawtooth doc		app-next/docs/architecture/fatigue-risk-sawtooth-model.md
FRMS integration		app-next/docs/architecture/frms-python-integration.md

Part 8 - Gaps and development directions

Gap | Implication

|-----|-----|

NHVR BFM not implemented | Provisional pack = WA + disclaimer only

Two prospective systems | TS 168h matrix vs Python TPMA register - unify or separate in UI
FRMS_ENGINE defaults to legacy | TPMA only when flag enabled
Matrix calibration | ADR 0003 notes L/C scales need fleet tuning
Camera BT bridge | Stub only - live fusion incomplete
Moving-vehicle rest | GPS heuristic, not definitive
Rule IP gate | .cursor/rules/time-rules-ip.mdc - compliance math needs owner approval to change

Product-safe extensions:

1. More prospective scenarios (Monte Carlo on planned legs, weather, roster)
2. Unify risk registers (168h matrix + TPMA breaches)
3. NHVR jurisdiction pack (separate BFM limits)
4. ML layer on TPMA features + camera (per frms-python-integration.md)
5. Barrier effectiveness modelling
6. Fleet-level aggregation (shift-lane projection already uses 5h rule)

Part 9 - Regulatory positioning

Claim | Supported?

|-----|-----|

WA Reg 184E retrospective checks on attested record | Yes (compliance engine)

ISO 31000 prospective coaching on future plans | Yes (risk register)

NHVR-approved EWD | No

NHVR FRMSc biomathematical certification | No

NHVR BFM rule pack | Provisional only (WA math + banner)

Explicit disclaimer: Risk scores and FRMS timelines are assurance and coaching tools. They do not replace compliance verdicts on logged time.

Document history

Date | Change

|-----|-----|

2026-07-08 | Initial technical overview for product development